Proto-Cuneiform Preprocessing

22/02/2024

Description

This file enables preprocessing of proto-cuneiform tablet transliterations, these can be downloaded from CDLI (https://cdli.mpiwg-berlin.mpg.de/search). The extraction of proto-cuneiform sign codings is done with the sign lists provided by CDLI. In their syntax repetitions of signs are indicated by a number followed by the sign type in parentheses, e.g. 1(N01), 3(N01), 5(N14), etc. This is here expanded to yield individual sign tokens, e.g. N01 N01 N01, etc. Note that signs which are followed by '#' (broken signs), and by '?' (questionable), are not included here.

Session Info

Give the session info (reduced).

```
## [1] "R version 4.2.2 Patched (2022-11-10 r83330)"
```

```
## [1] "x86_64-pc-linux-gnu"
```

Load Libraries

```
library(Hmisc)
```

```
## Loading required package: lattice
```

```
## Loading required package: survival
```

```
## Loading required package: Formula
```

```
## Loading required package: ggplot2
```

```
##
## Attaching package: 'Hmisc'
```

```
## The following objects are masked from 'package:base':
##
##     format.pval, units
```

```
library(stringr)
library(readr)
```

Load Transliteration and Sign List

Loading transliterations and sign lists from CDLI. Note that different datasets can be loaded by replacing e.g. "UrukV" to "UrukIV" in the file paths below to point to the respective folder. This also needs to be

changed for the output file!

```
# transliteration as text file with line breaks
translit.file = "~/Github/UpperPalCombinatorics/data/CDLI/UrukV/inscriptions.atf"
textlines <- scan(translit.file, what = "char", quote = "", comment.char = "",
                  encoding = "UTF-8", sep = "\n" , skip = 0)
# load transliteration as single character string
textstring <- read_file("~/Github/UpperPalCombinatorics/data/CDLI/UrukV/inscriptions.atf")
# load sign list
signlist.file = "~/Github/UpperPalCombinatorics/data/CDLI/UrukV/signlist.txt"
signlist <- scan(signlist.file, what = "char", quote = "", comment.char = "",
                  encoding = "UTF-8", sep = "\n" , skip = 0)
```

Process Transliteration

Get vector of IDs for tablets.

```
# the ampersand followed by P identifies a single tablet transliteration
tablet.id <- textlines[grep(pattern = "\\&P", textlines)]
# check length
length(tablet.id)
```

```
## [1] 141
```

```
# remove ampersand
tablet.id <- gsub(pattern = "\\&", "", tablet.id)
# get just the ID
tablet.id <- substr(tablet.id, 1, 7)
```

Extract and concatenate the transliterations.

```
# get chunks of text by ampersand plus "P" (at the beginning of each tablet)
textchunks <- str_split(textstring, pattern = "\\&P")
# unlist and remove first element (empty)
textchunks <- unlist(textchunks)[-1]
# remove the header
textchunks <- gsub("\\=.*qpc", "", textchunks)

# use sign list to extract only signs
# first some characters in the sign list need to be escaped
signlist <- escapeRegex(signlist)
# add white spaces around character strings in signlist
# this helps with only extracting full string matches
signlist <- paste0(signlist,' ')
# remove 'X' as this is a placeholder for unidentified signs
signlist <- signlist[!signlist == 'X ']

# run for loop to extract signs based on sign list
# initialize vectors
signs.orig.vec <- c()
signs.expand.vec <- c()
for (i in 1:length(textchunks)){
  # note that the order of signs is here sometimes changed,
  # as it follows the order of signs in sign list
  signs.orig <- unlist(str_extract_all(textchunks[i], paste(signlist, collapse = "|")))
```

```
  signs <- unlist(str_extract_all(textchunks[i], paste(signlist, collapse = "|")))
  # trim white spaces around signs
  signs.orig <- trimws(signs.orig)
  signs <- trimws(signs)
  # unravel the multiplication syntax to get multiple sign tokens
  try({ # if the processing fails for a certain row, there will be no output for this row,
  # but the try() function allows the loop to keep running
  for (j in 1:length(signs)){
    sign.syntax <- signs[j]
    # get only number
    remainder <- sub(pattern = "\\(.*\\)", "", sign.syntax)
      # if number then multiply with sign type, else
      # leave sign syntax as is
      if (!grepl("[^0-9]", remainder)){
        # get sign.type inside round brackets
        sign.type <- gsub(".*\\((.*)\\).*", "\\1", sign.syntax)
        # repeat given the number ("remainder" from above)
        sign.tokens <- rep(sign.type, remainder)
        # collaps into character string
        sign.tokens <- paste(sign.tokens, collapse = " ")
      } else {
        sign.tokens <- sign.syntax
      }
    signs[j] <- sign.tokens
  }}) # end of try() function
  # collapse the signs into one string
  signs <- paste(signs, collapse = " _ ")
  signs.orig <- paste(signs.orig, collapse = " _ ")
  signs.orig.vec <- c(signs.orig.vec, signs.orig)
  signs.expand.vec <- c(signs.expand.vec, signs)
}
```

```
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
```

```
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
```

```
# compile final data frame
df <- cbind(tablet.id, signs.orig.vec, signs.expand.vec)
head(df)
```

```
##      tablet.id signs.orig.vec
## [1,] "P005303" ""
## [2,] "P005304" "5(N14) _ 3(N14)"
## [3,] "P008560" "6(N14) _ 4(N14) _ 6(N14)"
## [4,] "P008673" "2(N14) _ 5(N14) _ 9(N14)"
## [5,] "P005067" "3(N14) _ 3(N01)"
## [6,] "P009401" ""
##      signs.expand.vec
## [1,] ""
## [2,] "N14 N14 N14 N14 N14 _ N14 N14 N14"
## [3,] "N14 N14 N14 N14 N14 N14 _ N14 N14 N14 N14 _ N14 N14 N14 N14 N14 N14"
## [4,] "N14 N14 _ N14 N14 N14 N14 N14 _ N14 N14 N14 N14 N14 N14 N14 N14 N14"
## [5,] "N14 N14 N14 _ N01 N01 N01"
## [6,] ""
```

Safe to file.

```
write.csv(df, file = "~/Github/UpperPalCombinatorics/data/CDLI/UrukV/signs_clean.csv", row.names = F)
```